

ANALISI GEOSTATISTICA DEI DATI AMBIENTALI – Livello intermedio (T28_25_3_2)

Francesco Pirotti

2025-11-29

Contents

1	ANALISI GEOSTATISTICA DEI DATI AMBIENTALI – Livello intermedio	2
1.1	Riferimenti utili	2
1.2	Obiettivo	2
1.3	Giornata 1	2
1.4	Giornata 2	2
1.5	Giornata 3	2
1.6	Dati di esempio	3
1.7	Analisi Esplorativa dei dati	5
1.8	Interpolazioni con metodo deterministico	6
1.9	Test autocorrelazione spaziale	7
1.10	Test tendenza	9
1.11	Analisi Esplorativa del Variogramma	10
1.12	Verifica di anisotropia	11
1.13	Modelli Multivariati - Co-kriging	12
1.14	Modello del variogramma dei residui	13
1.15	Interpolazioni geospaziali	14
1.16	Collocated Cokriging	16
1.17	Global vs local kriging	17
1.18	Block kriging	19
1.19	Kriging stratificato/indicator?	20
1.20	Kriging con trend con variabile discreta	21
1.21	Nota su interpretazione scale trasformate	24
1.22	Validazione	25
1.23	Indicatori di accuratezza	25
1.24	Confronto performance modelli diversi	26
1.25	Mappa degli errori standardizzati	26

1 ANALISI GEOSTATISTICA DEI DATI AMBIENTALI – Livello intermedio

1.1 Riferimenti utili

CRS - coordinate reference system

<https://r.geocompx.org/spatial-class#crs-intro>

<https://r-spatial.org/book/02-Spaces.html#sec-crs>

meuse dataset - <https://cloud.r-project.org/web/packages/gstat/vignettes/gstat.pdf>

veneto dataset in cartella “**dati**”

metodi di interpolazione geografica - <https://r-spatial.org/book/12-Interpolation.html>

1.1.0.1 Applied Spatial Data Analysis with R (Bivand et al.) - <https://gis.humboldt.edu/OLM/r/Spatial%20Analysis%20With%20R.pdf>

1.2 Obiettivo

Seguiremo il testo “Applied Spatial Data Analysis with R” di Bivand, Pebesma e Gómez-Rubio (vedi link sopra)

1.3 Giornata 1

- Ripasso basi teoriche, analisi esplorativa dei dati per verifiche presupposti al Kriging
- Richiamo sulle statistiche descrittive, distribuzioni, trasformazioni, sistemi di riferimento
- Richiamo geostatistica: variabili regionalizzate, autocorrelazione spaziale
- Il variogramma: definizione, semivarianza, sperimentale vs modello, legame con la continuità spaziale.

1.4 Giornata 2

- Calcolo dell'autocorrelazione (es. indice di Moran) e del variogramma sperimentale - dati del veneto - esercizio da consegnare
- Discussione: “Cosa ci dice il variogramma? Quali sono i punti deboli (campionamento, anisotropia, non stazionarietà)?”
- Interpolazione (kriging) e stima spaziale - cokriging, block kriging, collocated kriging, global vs local kriging, indicator kriging
- Introduzione alla validazione

1.5 Giornata 3

- Esercizi con tipologie diverse di Kriging
- Valutazione dell'incertezza
- Validazione (cross validation)

Table 1: Dataset *meuse*

x	y	cadmium	copper	lead	zinc	elev	dist	om	ffreq	soil	lime	landuse	dist.m
181072	333611	11.7	85	299	1022	7.909	0.0013580	13.6	1	1	1	Ah	50
181025	333558	8.6	81	277	1141	6.983	0.0122243	14.0	1	1	1	Ah	30
181165	333537	6.5	68	199	640	7.800	0.1030290	13.0	1	1	1	Ah	150
181298	333484	2.6	81	116	257	7.655	0.1900940	8.0	1	2	0	Ga	270
181307	333330	2.8	48	117	269	7.480	0.2770900	8.7	1	2	0	Ah	380

1.6 Dati di esempio

Useremo il dataset “meuse” e “veneto” e le librerie che vedete sotto

```
library(lattice)
library(tidyverse)
library(stats)
library(ggplot2)
library(terra)
library(gstat)
library(kableExtra)
library(sp)
data(meuse)
#class(meuse)

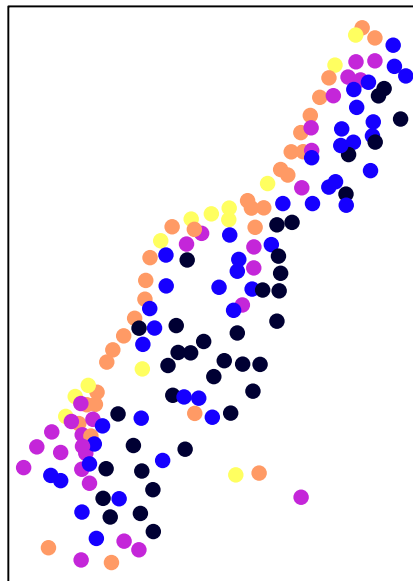
kable(head(meuse, 5), caption = "Dataset *meuse*") %>%
  kable_styling(full_width = FALSE, bootstrap_options = c("striped", "hover")) %>%
  kable_styling(latex_options = "scale_down")
```

Esercizio: provate a replicare quanto sotto con il dataset “Veneto” che trovate nel file CSV

pg 192 del testo [Applied Spatial Data Analysis with R (Bivand et al.) - <https://gis.humboldt.edu/OLM/r/Spatial%20Analysis%20With%20R.pdf>]

[Trovate i dati climatici del veneto in formato CSV nella cartella “dati”... il CRS dei dati del Veneto è il 3003 (Gauss Boaga Fuso Ovest)]

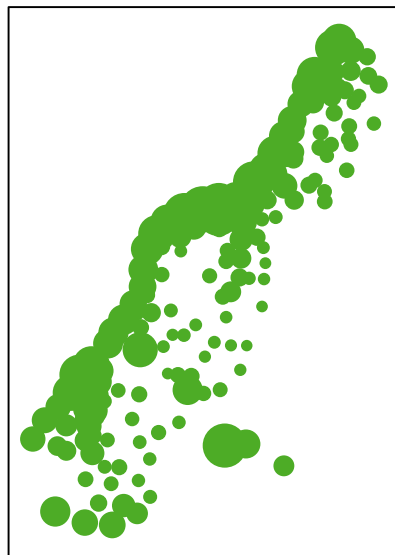
```
data(meuse)
coordinates(meuse) <- c("x", "y")
spplot(meuse, "zinc", do.log = T)
```



● [113,197.4]
 ● (197.4,344.9]
 ● (344.9,602.5]
 ● (602.5,1053]
 ● (1053,1839]

```
bubble(meuse, "zinc", do.log = T, key.space = "bottom")
```

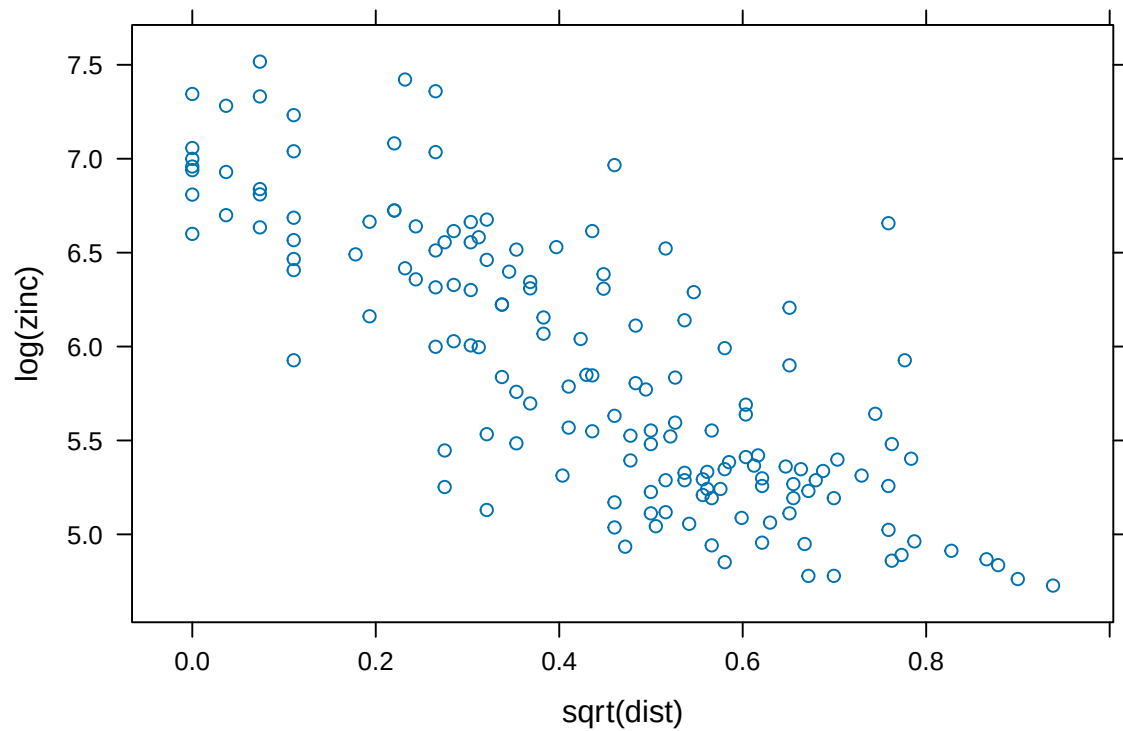
zinc



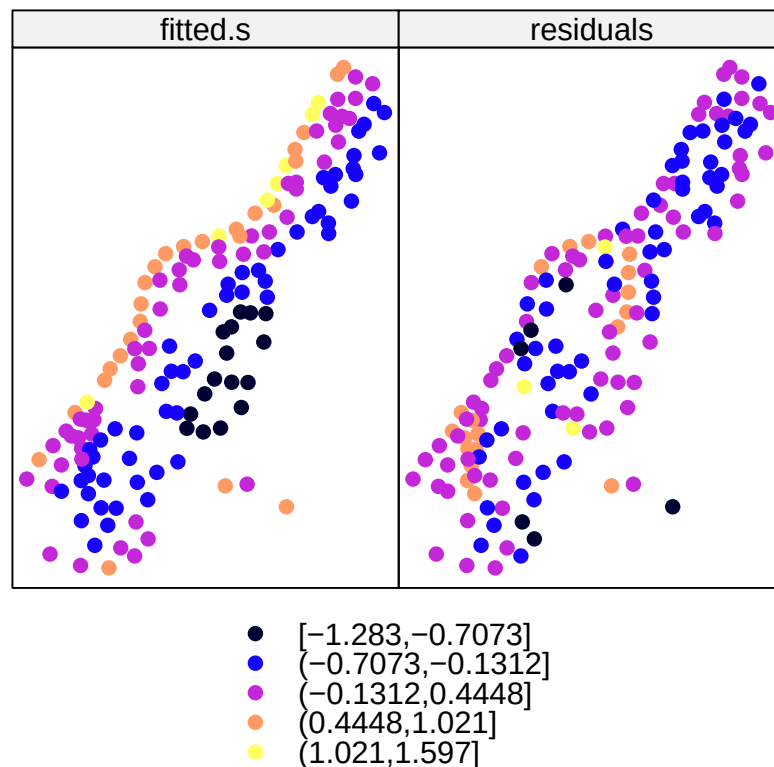
● 113
 ● 198
 ● 326
 ● 674.5
 ● 1839

1.7 Analisi Esplorativa dei dati

```
library(lattice)
xyplot(log(zinc) ~ sqrt(dist), as.data.frame(meuse))
```



```
zn.lm <- lm(log(zinc) ~ sqrt(dist), meuse)
meuse$fitted.s <- predict(zn.lm, meuse) - mean(predict(zn.lm, meuse))
meuse$residuals <- residuals(zn.lm)
spplot(meuse, c("fitted.s", "residuals"))
```



```
# spplot(meuse, c("fitted.s"))
# spplot(meuse, c("residuals"))
```

1.8 Interpolazioni con metodo deterministico

1.8.1 Inverso della distanza

Attenzione usiamo `idw` esprimendolo usando `gstat::idw` in quanto altri packages in R hanno la funzione `IDW` e può causare confusione ed errori

[Inverse-distance weighted smoothing of observations at irregular points](#)

(in package [spatstat.explore](#) in library /archivio/usr/local/lib/R/site-library)

[Simple, Ordinary or Universal, global or local, Point or Block Kriging, or simulation.](#)

(in package [gstat](#) in library /archivio/usr/local/lib/R/site-library)

```
Error in idw(zinc ~ 1, meuse, meuse.grid, idp = 2.5) :
  is.ppp(X) && is.marked(X) is not TRUE
```

```
library(gstat)
data(meuse.grid)
coordinates(meuse.grid) <- c("x", "y")
meuse.grid <- as(meuse.grid, "SpatialPixelsDataFrame")

idw.out <- gstat::idw(zinc ~ 1, meuse, meuse.grid, idp = 2.5)
```

```
## [inverse distance weighted interpolation]
```

1.8.2 Interpolazione lineare o non-lineare

```
zn.lm <- lm(log(zinc) ~ sqrt(dist), meuse)
meuse.grid$pred <- predict(zn.lm, meuse.grid)
meuse.grid$se.fit <- predict(zn.lm, meuse.grid, se.fit = TRUE)$se.fit
### oppure
meuse.tr2 <- krige(log(zinc) ~ 1, meuse, meuse.grid, degree = 2)
```

```
## [ordinary or weighted least squares prediction]
```

```
## oppure
lm(log(zinc) ~ I(x^2) + I(y^2) + I(x * y) + x + y, meuse)
```

```
##
## Call:
## lm(formula = log(zinc) ~ I(x^2) + I(y^2) + I(x * y) + x + y,
##     data = meuse)
##
## Coefficients:
## (Intercept)      I(x^2)      I(y^2)      I(x * y)          x          y
##  2.395e+04    8.575e-07    8.467e-07   -1.623e-06    2.279e-01   -2.684e-01
```

1.9 Test autocorrelazione spaziale

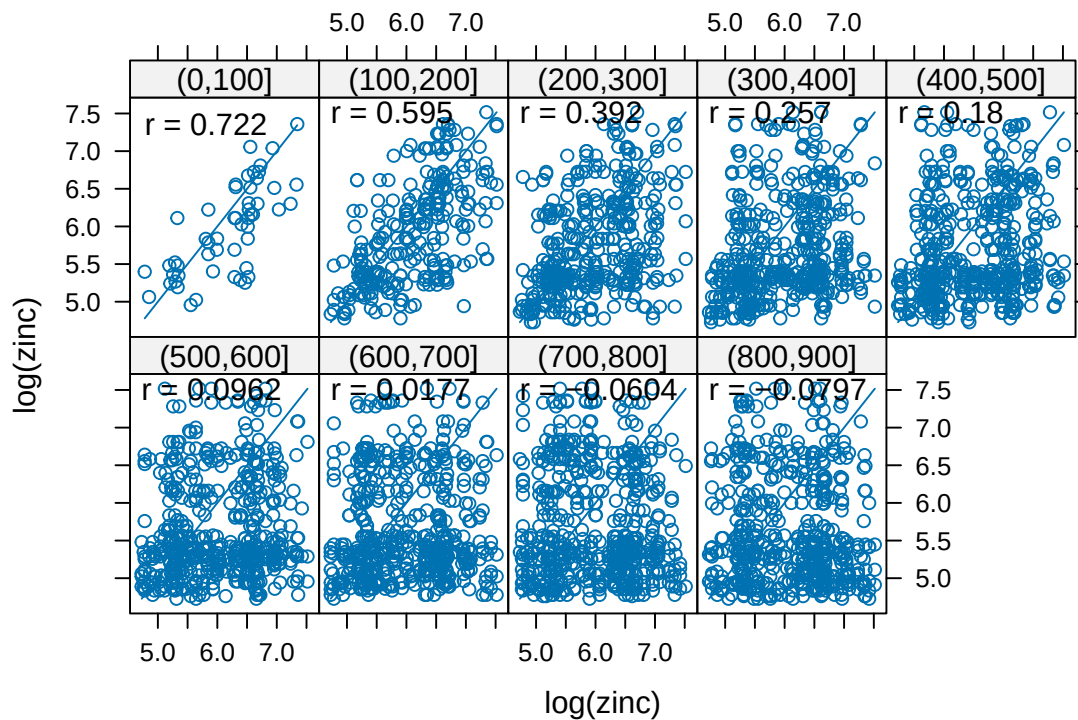
Vedi cap. 9.4 Applied Spatial Data Analysis with R - leggere attentamente per comprendere l'effetto di eventuali trend

Funzione hscat indicando i lag... applicare al dato "veneto" - che lag mettiamo?

Indice di Moran ci fornisce un test con significatività del risultato - attenzione, vedi sotto

```
hscat(log(zinc) ~ 1, meuse, (0:9) * 100)
```

lagged scatterplots



```
## ma come quantificare la correlazione spaziale? Indice di Moran -
suppressPackageStartupMessages(library(spdep))
nb <- dnearneigh(coordinates(meuse), 0, 3000)
lw <- nb2listw(nb, style = "W")

# Moran I
moran.test(log(meuse$zinc), lw)
```

```
##
## Moran I test under randomisation
##
## data: log(meuse$zinc)
## weights: lw
##
## Moran I statistic standard deviate = -0.73355, p-value = 0.7684
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic      Expectation      Variance
## -8.721268e-03      -6.493506e-03      9.223104e-06
```

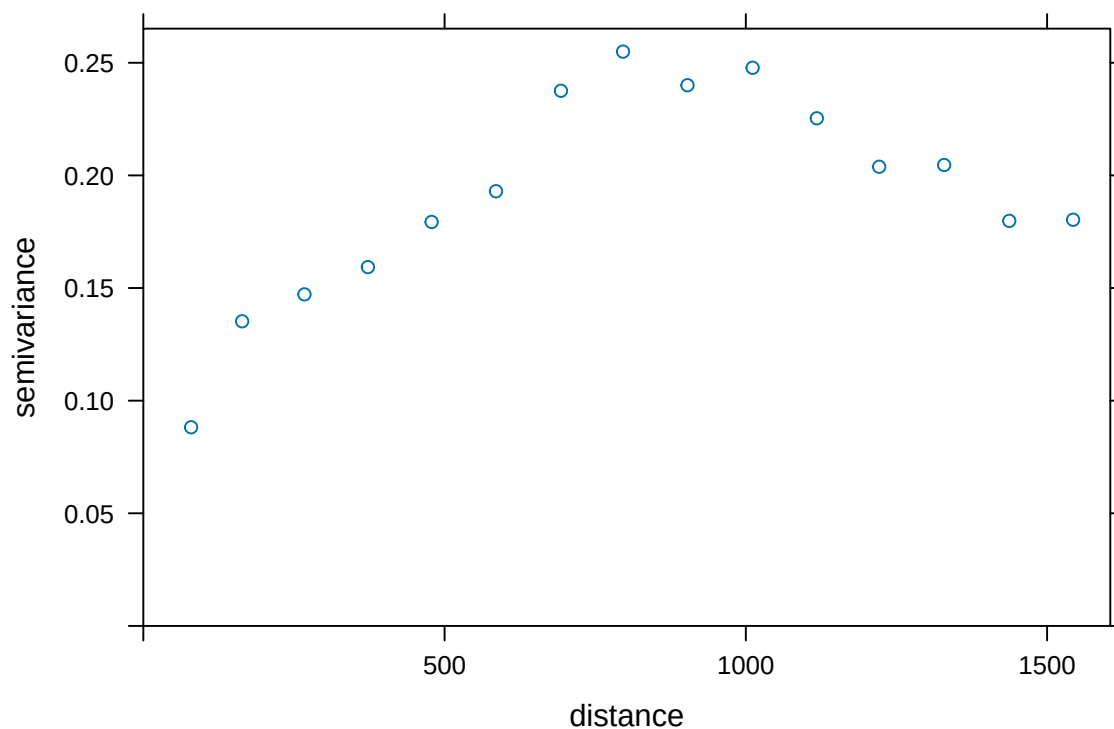
```
## ma sui residui mi dice non significativo, mentre nel variogramma si: come mai?
zn.lm <- lm(log(zinc) ~ sqrt(dist), meuse)
meuse@data$residuals <- zn.lm$residuals
moran.test(log(meuse$zinc), lw)
```

```
##
## Moran I test under randomisation
##
## data: log(meuse$zinc)
## weights: lw
```



```
##
## Moran I statistic standard deviate = -0.73355, p-value = 0.7684
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic      Expectation      Variance
##      -8.721268e-03      -6.493506e-03      9.223104e-06
```

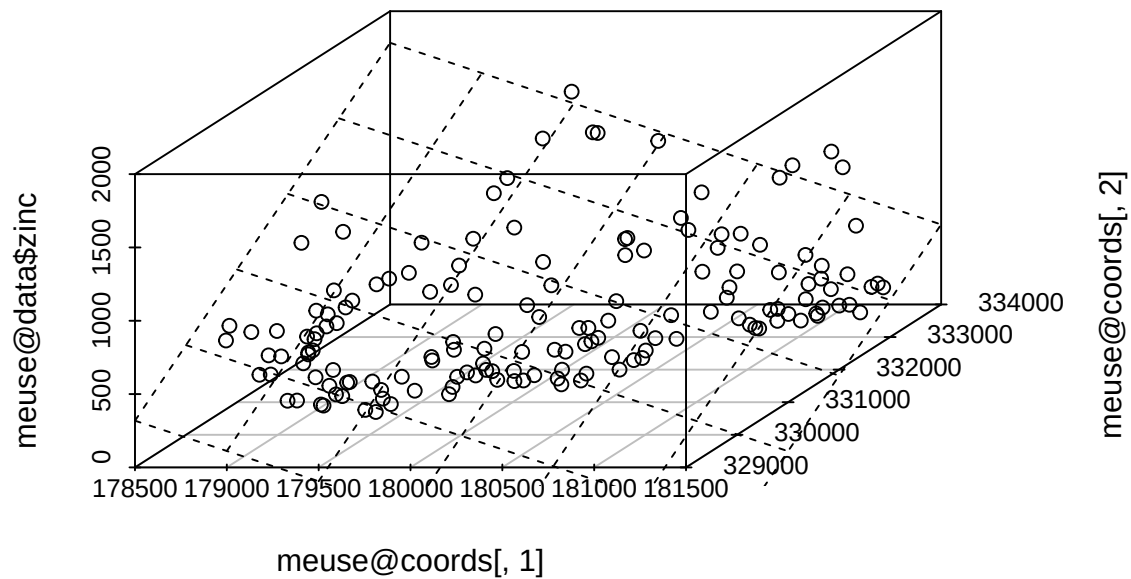
```
v <- variogram(residuals ~ 1, meuse)
plot(v)
```



1.10 Test tendenza

```
library(scatterplot3d)
# Aggiusto un modello di un piano nello spazio 2d
fit <- lm(meuse@data$zinc ~ meuse@coords[,1] + meuse@coords[,2])
scatplot <- scatterplot3d( meuse@coords[,1], meuse@coords[,2],
                           meuse@data$zinc, main="3D Scatterplot")
scatplot$plane3d(fit)
```

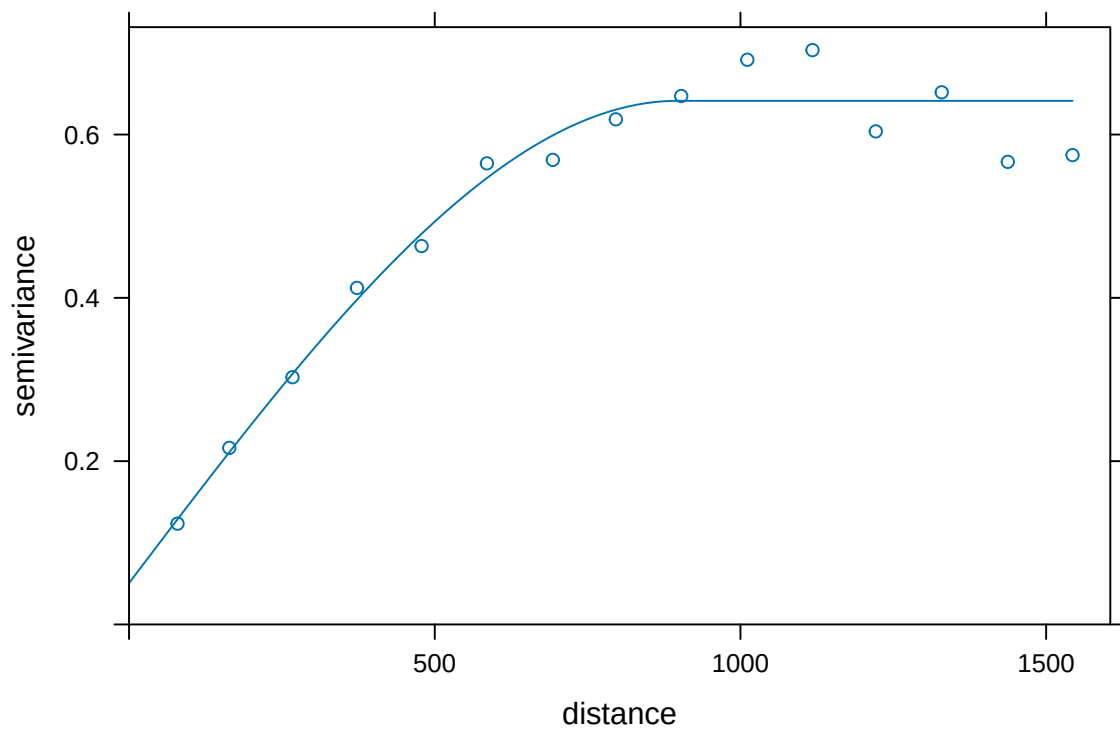
3D Scatterplot



1.11 Analisi Esplorativa del Variogramma

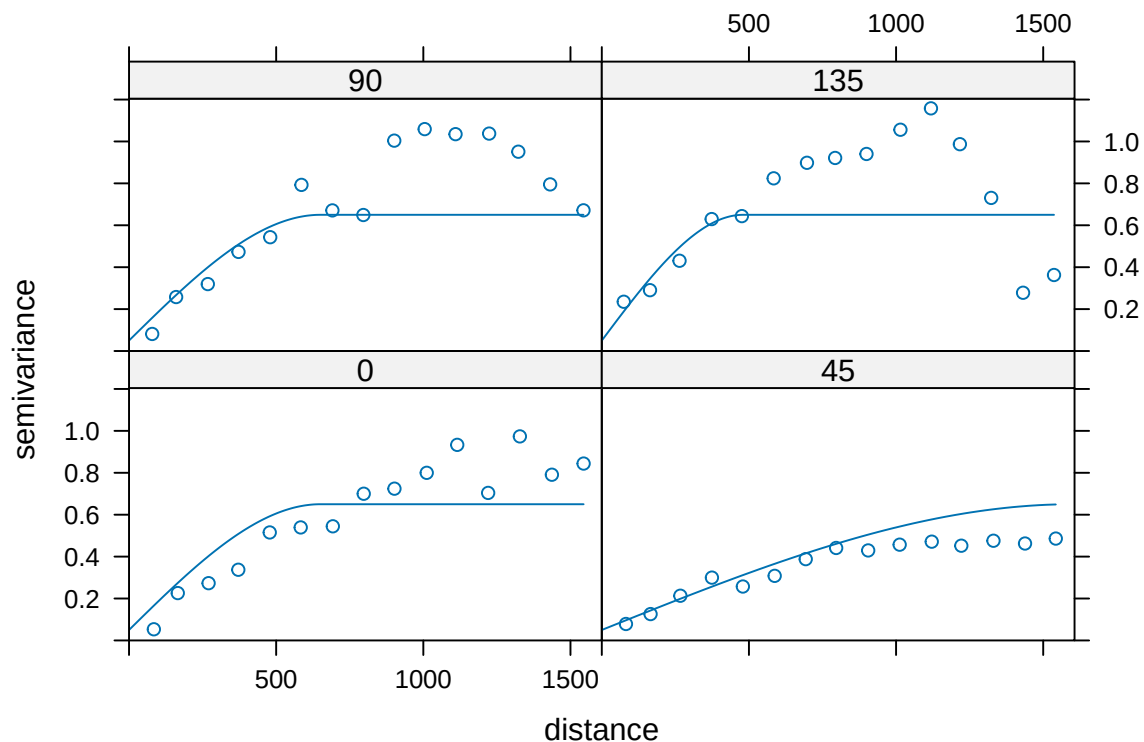
1.11.1 Modello del variogramma

```
## oppure classico variogramma
# attenzione qui fornisce TUTTE le combinazioni
# variogram(log(zinc) ~ 1, meuse, cloud = TRUE)
## library(geoR)
v <- variogram(log(zinc) ~ 1, meuse)
v.fit <- fit.variogram(v, vgm(1, "Sph", 800, 1))
plot(v, v.fit)
```



1.12 Verifica di anisotropia

```
v.dir <- variogram(log(zinc) ~ 1, meuse, alpha = (0:3) * 45)
## variogramma dove indichiamo che in direzione 45°
v.anis <- vgm(0.6, "Sph", 1600, 0.05, anis = c(45, 0.3))
plot(v.dir, v.anis)
```



NB possiamo variare il passo nel variogramma e la distanza max, ma dipende da quanta densità abbiamo i dati
plot(variogram(log(zinc) ~ 1, meuse, map = TRUE, cutoff = 1000, width = 100))

1.13 Modelli Multivariati - Co-kriging

I modelli multivariati considerano più variabili dipendenti, ovvero variabili coregionalizzate (o coregionalized variables) che sono variabili che vengono analizzate congiuntamente, perché misurate insieme e correlate tra loro nello spazio. L'idea è che una variabile possa "aiutare" a stimare un'altra, se condividono una struttura di dipendenza spaziale e se c'è correlazione.

```
cor(as.data.frame(meuse)[c("cadmium", "copper", "lead", "zinc")])
```

```
##          cadmium    copper      lead      zinc
## cadmium 1.0000000 0.9254499 0.7989466 0.9162139
## copper   0.9254499 1.0000000 0.8183069 0.9082695
## lead     0.7989466 0.8183069 1.0000000 0.9546913
## zinc     0.9162139 0.9082695 0.9546913 1.0000000
```

Ho quattro variabili (Cd, Cu, Pb, Zn), log-trasformate, misurate negli stessi punti. Voglio modellarle insieme per il co-kriging

```
g <- gstat(NULL, "logCd", log(cadmium) ~ 1, meuse)
g <- gstat(g, "logCu", log(copper) ~ 1, meuse)
g <- gstat(g, "logPb", log(lead) ~ 1, meuse)
g <- gstat(g, "logZn", log(zinc) ~ 1, meuse)
```

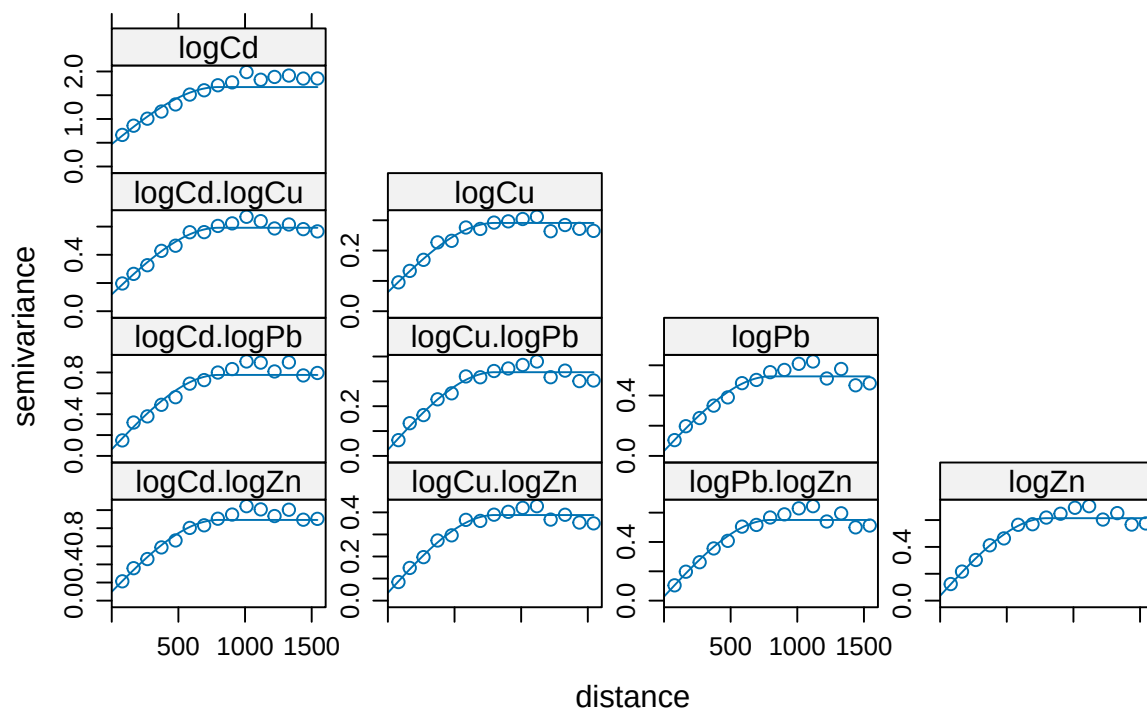
Si calcolano variogrammi e cross-variogrammi, 4 variogrammi diretti (Cd-Cd, Cu-Cu, Pb-Pb, Zn-Zn), 6 cross-variogrammi (Cd-Cu, Cd-Pb, ... etc.)

In totale 10 curve che descrivono: la variabilità spaziale di ogni elemento, come questi co-variano nello spazio

```
vm <- variogram(g)
```

Si adatta/stima un LMC (Linear Model of Coregionalization) - Usa un'unica struttura variografica (sferica, range 800) e costruisce un modello multivariato valido per tutte le variabili. Ovvero: prende lo stesso modello di variogramma (sferico) per ogni elemento, e stima, un sill per ogni variabile, un cross-sill per ogni coppia e mette questi sill e cross-sill in una matrice di coregionalizzazione. La matrice deve essere semidefinita positiva (garantisce che il modello multivariato sia valido)

```
vm.fit <- fit.lmc(vm, g, vgm(1, "Sph", 800, 1))
plot(vm, vm.fit)
```



Adesso che ho stimato il modello posso interpolare (vedi sezione su Cokriging - modelli su multivariate)

1.14 Modello del variogramma dei residui

```
f <- log(zinc) ~ sqrt(dist)
vt <- variogram(f, meuse)
vt.fit <- fit.variogram(vt, vgm(1, "Exp", 300, 1))
vt.fit
```

```
##   model      psill    range
## 1  Nug 0.05712231  0.0000
## 2  Exp 0.17641559 340.3201
```

```
## confronta con il modello log(zinc) ~ 1

## si possono utilizzare GLS
g.wls <- gstat(NULL, "log-zinc", f, meuse, model = vt.fit, set = list(gls = 1))
## differenze tra i due variogrammi
## (variogram(g.wls)$gamma - vt$gamma)/mean(vt$gamma)
```

1.15 Interpolazioni geospaziali

1.15.1 Kriging “semplice”

nota che viene utilizzato il tipo di kriging in funzione dei dati di input

```
lz.sk <- krige(log(zinc) ~ 1, meuse, meuse.grid, v.fit, beta = 5.9)
```

```
## [using simple kriging]
```

1.15.2 Kriging ordinario

```
lz.ok <- krige(log(zinc) ~ 1, meuse, meuse.grid, v.fit)
```

```
## [using ordinary kriging]
```

1.15.3 Kriging universale

Solitamente utilizza le coordinate ed una polinomiale per catturare la variabilità che è funzione della posizione. Utilizza dunque informazioni note in tutte le posizioni di previsione.

```
f <- log(zinc) ~ poly(x,y)
vt <- variogram(f, meuse)
vt.fit <- fit.variogram(vt, vgm(1, "Exp", 300, 1))
vt.fit
```

```
##   model      psill   range
## 1   Nug 0.05029722  0.0000
## 2   Exp 0.47614253 555.3591
```

```
lz.uk <- krige(f, meuse, meuse.grid, vt.fit)
```

```
## [using universal kriging]
```

1.15.4 Kriging con deriva esterna (EDK - external drift kriging)

Un tipo specifico di UK dove la tendenza della variabile da prevedere è modellata in funzione di un predittore esterno, sempre noto in tutto lo spazio del campo di calcolo.

```
lz.edk <- krige(log(zinc) ~ sqrt(dist), meuse, meuse.grid, vt.fit)
```

```
## [using universal kriging]
```

1.15.5 Cokriging - modelli su multivariate

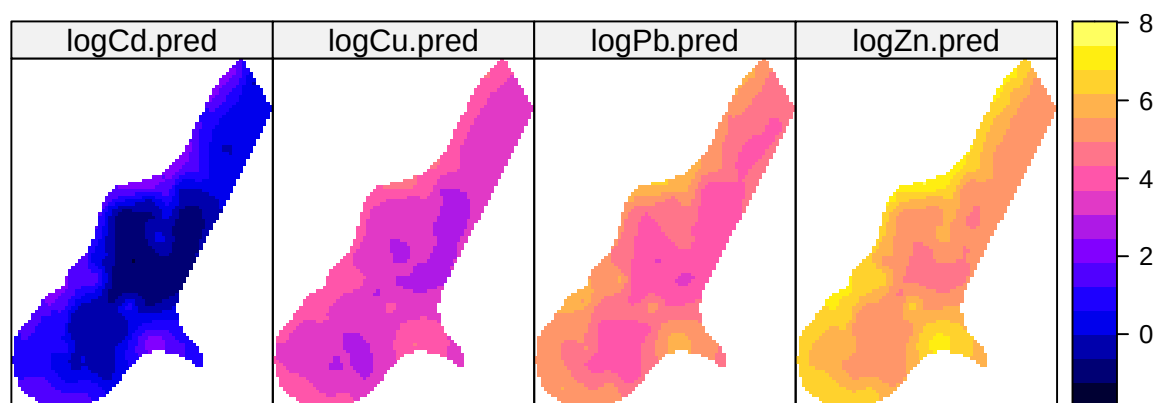
Applico il modello adattato nella sezione Modelli Multivariati - Co-kriging

```
cok.maps <- predict(vm.fit, meuse.grid)
```

```
## Linear Model of Coregionalization found. Good.  
## [using ordinary cokriging]
```

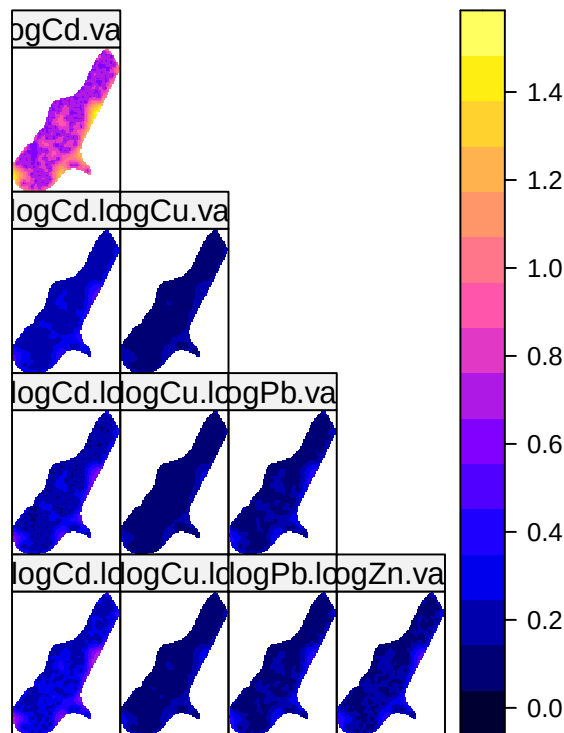
Posso disegnare le mappe delle stime spazializzate sulla griglia.

```
# spplot(cok.maps)  
spplot(cok.maps[ , grep("pred$", names(cok.maps))])
```



Una specie di “mappa dell’incertezza multivariata”. Mostra: varianze di predizione (una mappa per variabile) + covarianze tra variabili

```
spplot.vcov(cok.maps)
```



1.16 Collocated Cokriging

Due condizioni differenti quando abbiamo una variabile indipendente potenzialmente correlata con la variabile dipendente (che vogliamo stimare) che è disponibile sia nei punti campione sia nei punti che vogliamo stimare (la griglia). Come nel caso della distanza dai corsi d'acqua nel caso Meuse o ad esempio la quota nel caso di variabili climatiche del Veneto...

```
#Crea un oggetto gstat per log(zinc) nel dataset meuse.
g.cc <- gstat(NULL, "log.zinc", log(zinc) ~ 1, meuse, model = v.fit)
# v.fit è il modello di variogramma per log(zinc) usato prima.
# ~1 indica kriging ordinario / media costante.
# A questo punto hai un modello geostatistico univariato
meuse.grid$distn <- meuse.grid$dist - mean(meuse.grid$dist) + mean(log(meuse$zinc))
# Crea una nuova variabile distn sulla griglia di predizione.
# È centrata e scalata per avere la stessa media di log(zinc).
# L'idea: usare questa variabile come secondaria nel cokriging

##### # Adeguamento del variogramma per distn
vd.fit <- v.fit
# Copia il variogramma originale v.fit.
# Modifica il sill (psill) proporzionalmente al rapporto tra varianze della nuova variabile e dello
# Questo genera un variogramma sintetico per distn coerente con log(zinc).
vov <- var(meuse.grid$distn)/var(log(meuse$zinc))
vd.fit$psill <- v.fit$psill * vov

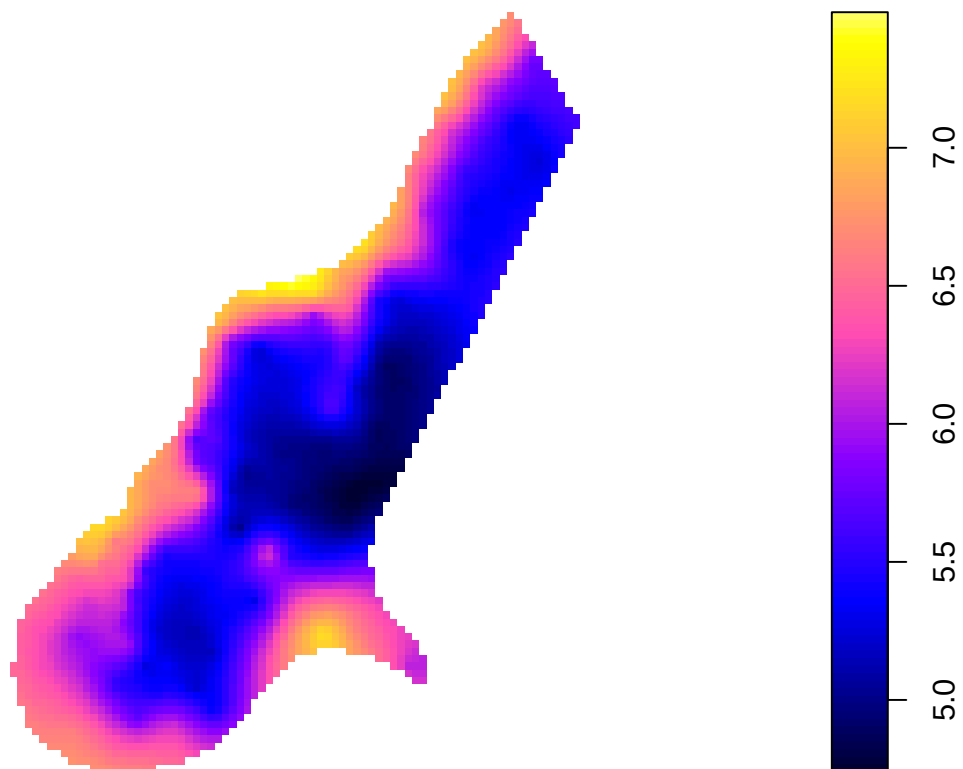
# Aggiunge distn come nuova variabile gstat
g.cc <- gstat(g.cc, "distn", distn ~ 1, meuse.grid, nmax = 1, model = vd.fit, merge = c("log.zinc",
# nmax = 1 significa considerare solo il vicino più prossimo per la variabile secondaria
# merge = c("log.zinc", "distn") combina le strutture dati per il cokriging
```



```
##### Variogramma incrociato per il cokriging
vx.fit <- v.fit
vx.fit$psill <- sqrt(v.fit$psill * vd.fit$psill) * cor(meuse$dist, log(meuse$zinc))
g.cc <- gstat(g.cc, c("log.zinc", "distn"), model = vx.fit)
x <- predict(g.cc, meuse.grid)
```

```
## Intrinsic Correlation found. Good.
## [using ordinary cokriging]
```

```
# Costruisce il variogramma incrociato tra log.zinc e distn.
# Questa è l'assunzione del modello lineare di coregionalizzazione (LMC).
# Rappresenta la correlazione spaziale tra variabile primaria e secondaria.
plot(x)
```



1.17 Global vs local kriging

Global kriging stima valori spaziali assumendo una struttura di covarianza su tutta l'area (trend globale), usa dunque tutte le osservazioni mentre local kriging adatta i pesi e la struttura di covarianza a vicinanze limitate, catturando variazioni spaziali locali. Globale può diventare pesante computazionalmente

```
# Local kriging: nmax = numero massimo di vicini considerati
# Qui consideriamo solo i 5 vicini più prossimi per ogni predizione
lk <- gstat(NULL, "log.zinc", log(zinc) ~ 1, meuse, model=v.fit, nmax=5)
# Stima
pred <- predict(lk, meuse.grid)
```

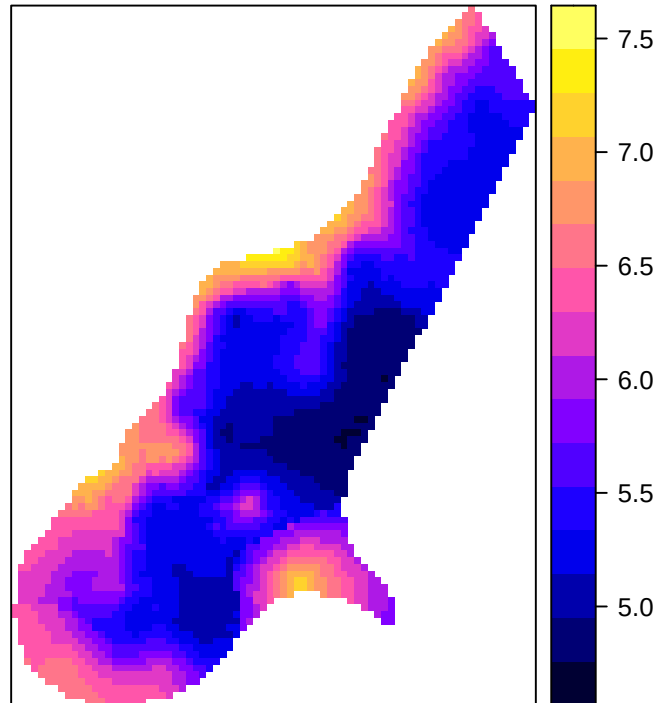
```
## [using ordinary kriging]
```

```

pred@data$log.zinc.sd <- pred@data$log.zinc.var^0.5
# Plot
spplot(pred, "log.zinc.pred", main="Local Ordinary Kriging Stima (nmax=5)")

```

Local Ordinary Kriging Stima (nmax=5)

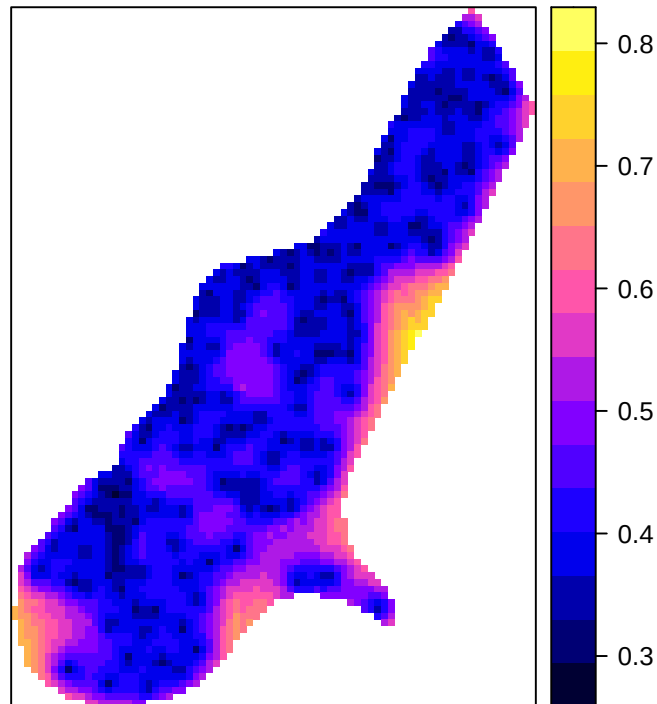


```

spplot(pred, "log.zinc.sd", main="Local Ordinary Kriging Incertezza (Dev. Std.) (nmax=5)")

```

Local Ordinary Kriging Incertezza (Dev. Std.) (nmax=5)



1.18 Block kriging

Le osservazioni $Z(s_i)$ sono considerate come misurate in punti specifici. Il kriging di un valore con dimensione fisica pari a quella delle osservazioni si chiama point kriging (kriging puntuale). Al contrario, il block kriging stima medie su aree o volumi più grandi.

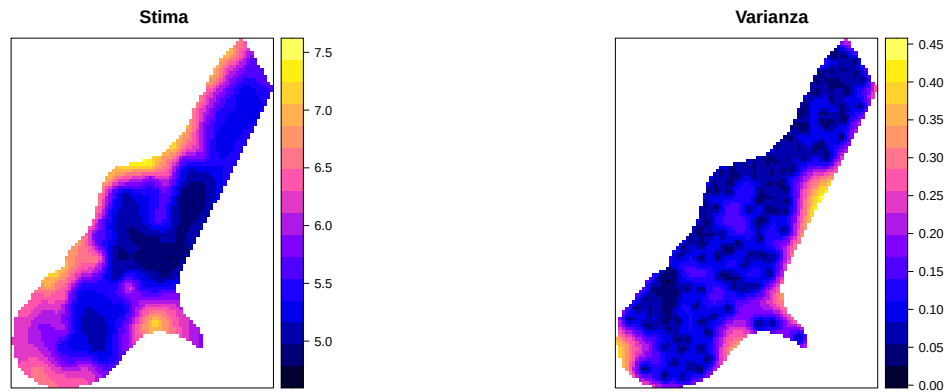
Il point kriging di solito mostra errori di stima più elevati. Questo avviene perché le osservazioni singole presentano maggiore variabilità.

Quando invece si stimano medie su aree più estese (block kriging), gran parte della variabilità interna ai blocchi si media, e quindi le medie dei blocchi hanno errori di stima più bassi, pur mantenendo visibili i pattern spaziali, purché i blocchi non siano troppo grandi.

```
lz.ok <- krige(log(zinc) ~ 1, meuse, meuse.grid, v.fit, block = c(40, 40))
```

```
## [using ordinary kriging]
```

```
spplot(lz.ok["var1.pred"], main="Stima")  
spplot(lz.ok["var1.var"], main="Varianza" )
```



1.19 Kriging stratificato/indicator?

Quando è disponibile una variabile **discreta (categoriche)** che suddivide l'area di interesse in un certo numero di aree disgiunte, ad esempio classi di uso del suolo o qualche altro fattore, potremmo voler applicare il kriging separatamente per le diverse classi. Questo approccio si chiama stratified kriging (kriging stratificato).

Il motivo per applicare il kriging per classe è che la struttura di covarianza (semivariogramma) può essere diversa tra le classi. A differenza del universal kriging con un predittore categorico, in questo caso non si assume alcuna correlazione tra i residui delle diverse classi. Esempio: temperatura in zone urbane vs temperatura in zone non urbane.

```
library(sp)
# Interpolazione iniziale della variabile categoriale part.a
meuse$part.a <- idw(part.a ~ 1, meuse.grid, meuse, nmax = 1)$var1.pred
```

```
## [inverse distance weighted interpolation]
```

```
# [interpolazione ponderata per distanza inversa]
# Qui qualsiasi interpolazione va bene, perché stiamo praticamente usando
# il **primo vicino più prossimo** come predittore.

# Approccio più robusto: usare overlay per assegnare valori della griglia
meuse$part.a <- over(meuse, meuse.grid)$part.a
# Questo metodo ignora eventuali osservazioni non coperte dalla griglia.

# Ora possiamo eseguire il kriging per ciascun sottodominio,
# memorizzare i risultati in x1 e x2, e poi unirli con rbind
# nella loro rappresentazione non spaziale.

# Kriging ordinario per il primo sottodominio (part.a == 0)
x1 <- krige(log(zinc) ~ 1,
  meuse[meuse$part.a == 0, ],
  meuse.grid[meuse.grid$part.a == 0, ],
  model = vgm(0.548, "Sph", 900, 0.0654),
  nmin = 20, nmax = 40, maxdist = 1000)
```

```
## [using ordinary kriging]
```

```
# [usando kriging ordinario]
```

```
# Kriging ordinario per il secondo sottodominio (part.a == 1)
```

```
x2 <- krige(log(zinc) ~ 1,
            meuse$part.a == 1, ],
            meuse.grid[meuse.grid$part.a == 1, ],
            model = vgm(0.716, "Sph", 900),
            nmin = 20, nmax = 40, maxdist = 1000)
```

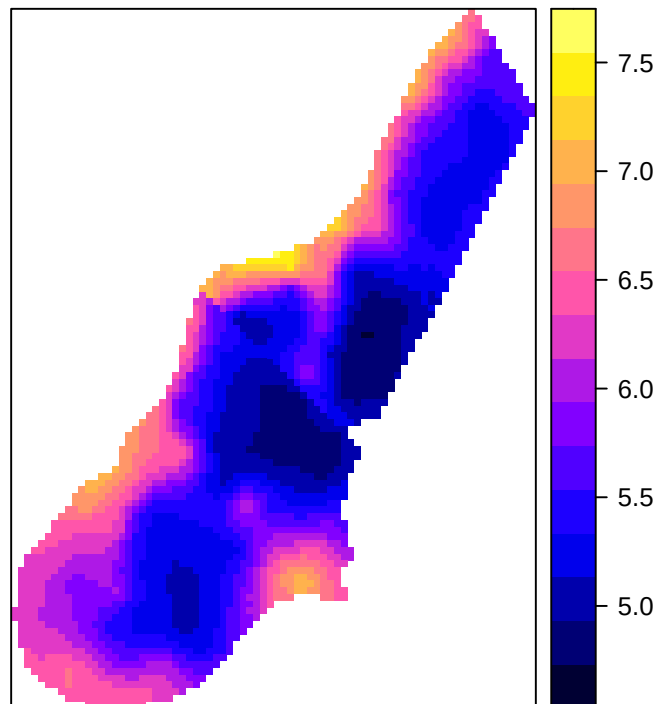
```
## [using ordinary kriging]
```

```
# [usando kriging ordinario]
```

```
# Unione dei risultati in un unico dataset
lz.stk <- rbind(as.data.frame(x1), as.data.frame(x2))
coordinates(lz.stk) <- c("x", "y")
lz.stk <- as(lz.stk, "SpatialPixelsDataFrame")

# Visualizzazione dei risultati di stratified kriging
spplot(lz.stk["var1.pred"], main = "Predizioni con kriging stratificato")
```

Predizioni con kriging stratificato



1.20 Kriging con trend con variabile discreta

Diverso da indicator kriging ... modella il trend come dipendente dalla categoria soil, e poi stima un **unico** variogramma dei residui; indicator kriging divide i variogrammi in funzione dei punti presenti per ogni classe.

```
library(sp)
library(gstat)

data(meuse)
```

```

coordinates(meuse) <- ~x+y

# Variogramma dei residui
v <- variogram(log(zinc) ~ soil, meuse)
v.fit <- fit.variogram(v, vgm(0.6, "Sph", 900, 0.1))

# Universal kriging con soil come covariata
g.uk <- gstat(formula = log(zinc) ~ soil, data = meuse, model = v.fit)

# Griglia di predizione
data(meuse.grid)
coordinates(meuse.grid) <- ~x+y
gridded(meuse.grid) <- TRUE

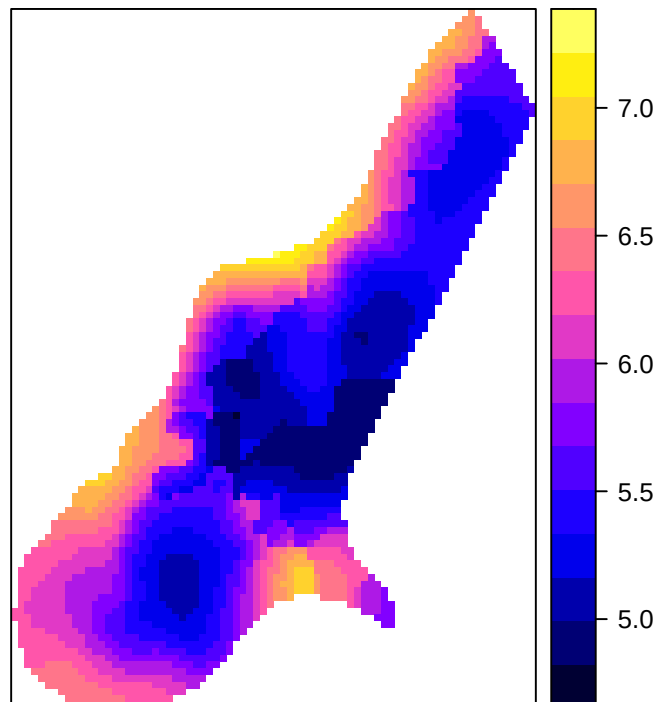
# Predizione
pred <- predict(g.uk, meuse.grid)

## [using universal kriging]

# Plot
# spplot(meuse.grid, c("soil"))
spplot(pred["var1.pred"], main="Universal kriging con soil come covariata")

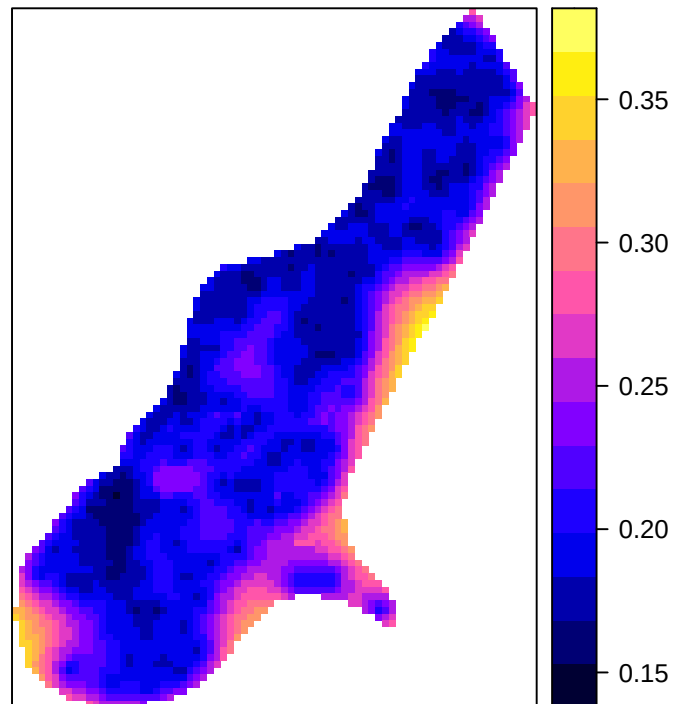
```

Universal kriging con soil come covariata



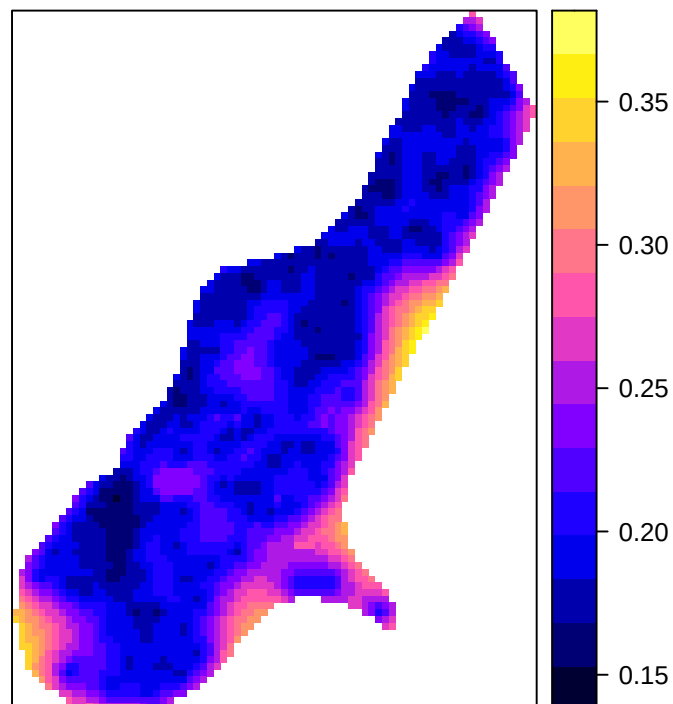
```
spplot(pred["var1.var"], main="Universal kriging con soil come covariata Varianza")
```

Universal kriging con soil come covariata Varianza



```
spplot(pred["var1.var"], main="Universal kriging con soil come covariata Varianza")
```

Universal kriging con soil come covariata Varianza



1.21 Nota su interpretazione scale trasformate

A volte si trasforma la variabile dipendente con log o altro. Attenzione nel voler tornare alla scala lineare.

```
library(gstat)
library(sp)

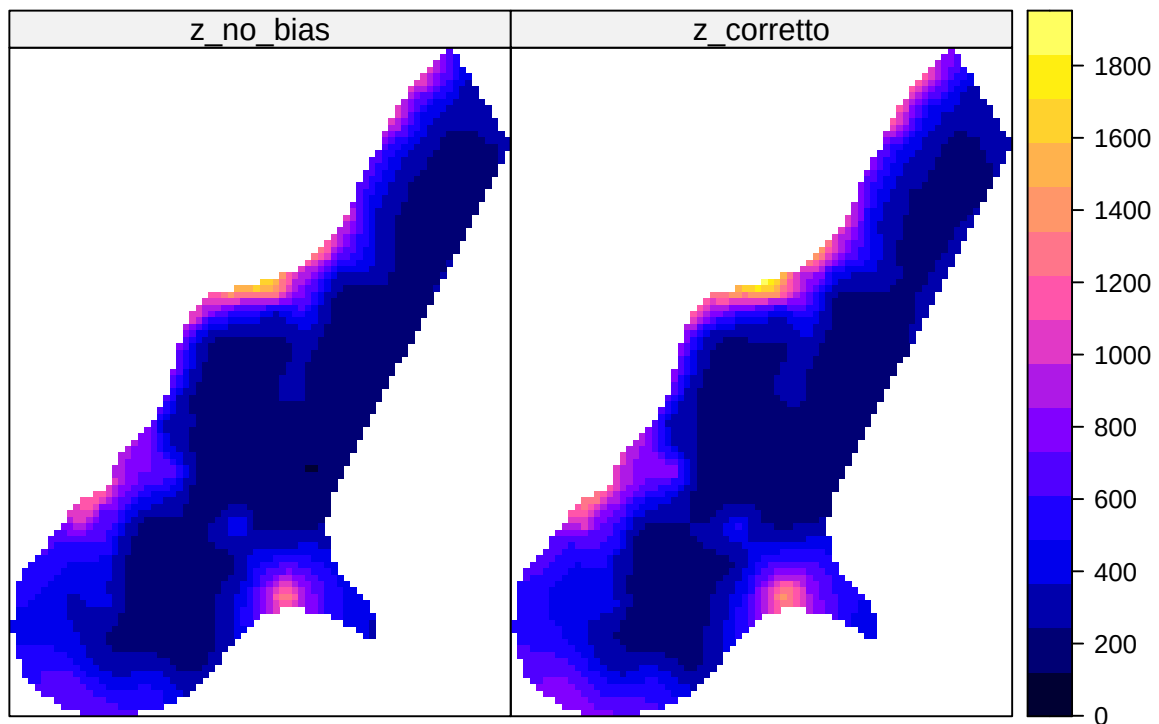
# modello in log
v.fit <- variogram(log(zinc) ~ 1, meuse)
v.fit <- fit.variogram(v.fit, vgm("Sph"))

# kriging in scala log
kr <- krige(log(zinc) ~ 1, meuse, meuse.grid, model = v.fit)
```

```
## [using ordinary kriging]
```

```
# back-transform senza correzione (Sbagliato)
kr$z_no_bias <- exp(kr$var1.pred)

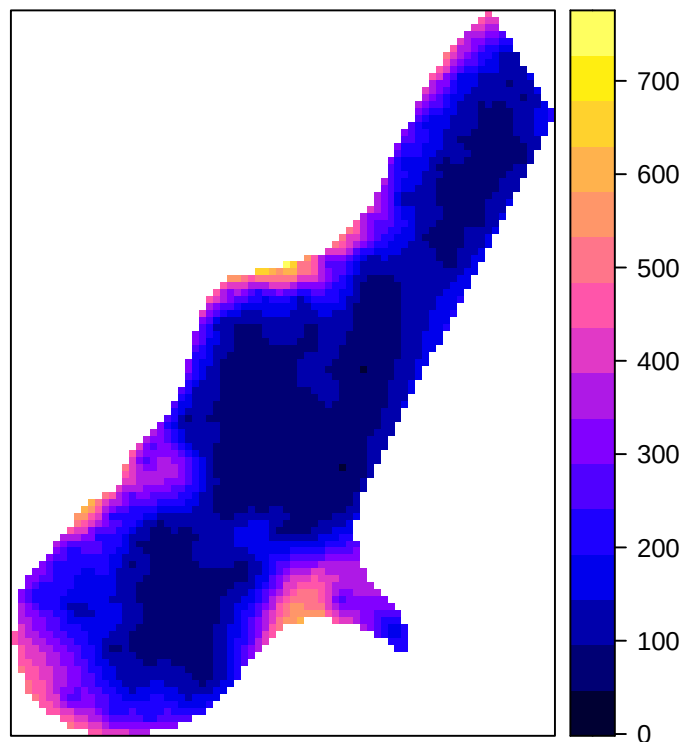
# back-transform con correzione (Corretto)
kr$z_corretto <- exp(kr$var1.pred + kr$var1.var / 2)
spplot(kr, c("z_no_bias", "z_corretto"))
```



```
#anche la varianza!
# calcolo della varianza back-transformata
kr$z_var_corretta <- exp(2*kr$var1.pred + kr$var1.var) * (exp(kr$var1.var) - 1)

# deviazione standard in scala originale
kr$z_sd <- sqrt(kr$z_var_corretta)

spplot(kr, c("z_sd"))
```

1.22 Validazione

La varianza mi stima l'affidabilità, ma una vera validazione richiede di confrontare il risultato con dati oggettivi misurati. Unico modo è quello di separare il dataset in un set di misure da usare per il modello ed un set misure da usare per la validazione. Importante che i due set siano entrambi rappresentativi della distribuzione (non biased).

cross-validazione: leave-one-out o k-fold. **Attenzione** che “leave-one-out” richiede molto tempo in quanto esegue krige tante volte quanti sono i punti!

```
cv <- krige.cv(log(zinc) ~ 1, meuse, model = v.fit)
```

1.23 Indicatori di accuratezza

Alla fine devo avere degli indicatori per confrontare “quale metodo funziona meglio”

Vedi codice in script R giorno3.meuse.R

```
# Errori:
ME <- mean(cv$residual)
RMSE <- sqrt(mean(cv$residual^2)) ### attenzione, NON sd(cv$residual)

# Residui standardizzati
MSDR <- mean(cv$zscore^2)

results <- data.frame(ME, RMSE, MSDR)
print(results)
```

```
##           ME          RMSE          MSDR
## 1 -2.059014e-05 0.3918018 0.8185451
```

1.24 Confronto performance modelli diversi

```
m1 <- krige.cv(log(zinc) ~ 1, meuse, model = v.fit)
m2 <- krige.cv(log(zinc) ~ sqrt(dist), meuse, model = v.fit)
m3 <- krige.cv(log(zinc) ~ soil, meuse, model = v.fit)

compare <- data.frame(
  Model = c("OK", "UK-dist", "UK-soil"),
  ME = c(mean(m1$residual), mean(m2$residual), mean(m3$residual)),
  RMSE = c(sqrt(mean(m1$residual^2)), sqrt(mean(m2$residual^2)), sqrt(mean(m3$residual^2))),
  MSDR = c(mean(m1$zscore^2), mean(m2$zscore^2), mean(m3$zscore^2))
)

compare
```

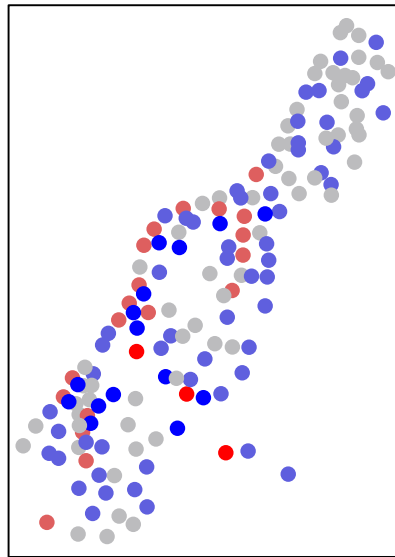
```
##      Model      ME      RMSE      MSDR
## 1      OK -2.059014e-05 0.3918018 0.8185451
## 2 UK-dist -3.965200e-03 0.3771411 0.7597823
## 3 UK-soil  6.021505e-04 0.3927407 0.8231116
```

Vedi codice in script R `giorno3.meuse.R` per una procedura che raccoglie i modelli e li valida tramite una funzione

1.25 Mappa degli errori standardizzati

```
cv_sp <- cv
#hist(cv@data$residual)
spplot(cv_sp, "zscore",
  col.regions = colorRampPalette(c("blue", "grey", "red"))(100),
  at = seq(-4, 4, length = 100),
  main = "Standardized residuals (z-score)")
```

Standardized residuals (z-score)



● [-2.24,-1.167]
● (-1.167,-0.09459]
● (-0.09459,0.978]
● (0.978,2.051]
● (2.051,3.123]

NB sempre la regola se vuoi tornare alla scala originale non log-trasformata

```
library(sp)
library(gstat)
data(meuse)
coordinates(meuse) = ~x+y

# Variogramma
v <- variogram(log(zinc) ~ 1, meuse)
v.fit <- fit.variogram(v, vgm("Sph"))

# Kriging su scala log
cv <- krige.cv(log(zinc) ~ 1, meuse, model = v.fit)
# predizione log e varianza
yhat <- cv$var1.pred
sigma2 <- cv$var1.var

# predizione back-log (bias correction)
z_pred <- exp(yhat + sigma2/2)

# residui back-log
z_obs <- meuse$zinc
resid_orig <- z_obs - z_pred

# deviazione standard back-log
sd_orig <- sqrt(exp(2*yhat + sigma2) * (exp(sigma2) - 1))

# z-score back-log
zscore_orig <- resid_orig / sd_orig

# aggiungo al dataset
```

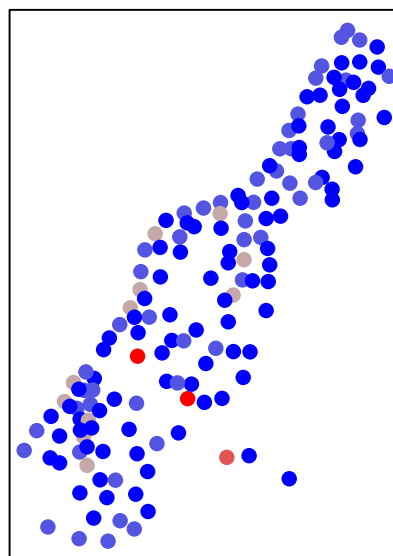
```
meuse$resid_backlog <- resid_orig
meuse$zscore_backlog <- zscore_orig

# Controllo
head(meuse@data[, c("zinc", "resid_backlog", "zscore_backlog")])
```

```
##   zinc resid_backlog zscore_backlog
## 1 1022      69.77428    0.1644587
## 2 1141     192.88121    0.4640965
## 3  640      45.33376    0.1701965
## 4  257    -213.54837   -0.8942327
## 5  269     -20.98095   -0.1653129
## 6  281      25.77188    0.1974802
```

```
spplot(meuse,
       "zscore_backlog",
       at = seq(-2, 2, length.out = 10), # colour breaks
       col.regions = colorRampPalette(c("blue", "grey", "red"))(10) ,
       main = "Standardized residuals (z-score)")
```

Standardized residuals (z-score)



- [-1.529,-0.1204]
- (-0.1204,1.289]
- (1.289,2.698]
- (2.698,4.107]
- (4.107,5.516]